

电气与电子工程学院

School of Electrical and Electronic Engineering

电子电路综合设计II

第四单元 用可编程逻辑器件制作红外计数器III

可编程逻辑器件的Verilog描述设计方法及验证

石 丹

一、《用可编程逻辑器件制作红外计数器Ⅱ》实验内容和要求

(1) 硬件描述语言简介

硬件描述语言（Hardware Description Language，HDL）是电子系统硬件行为描述、结构描述、数据流描述的语言。利用这种语言，数字电路系统的设计可以从顶层到底层（从抽象到具体）逐层描述自己的设计思想，用一系列分层次的模块来表示极其复杂的数字系统。然后，利用电子设计自动化（EDA）工具，逐层进行仿真验证，再把其中需要变为实际电路的模块组合，经过自动综合工具转换到门级电路网表。接下去，再用专用集成电路 ASIC 或现场可编程门阵列 FPGA 自动布局布线工具，把网表转换为要实现的具体电路布线结构。

与原理图输入方式相比，使用硬件描述语言设计电路的最明显的特点是避开了器件的功能和结构，无需检索和掌握众多逻辑器件的使用方法，只需要从逻辑行为上进行描述。

Verilog HDL是硬件描述语言的一种，用于数字电子系统设计。该语言允许设计者进行各种级别的逻辑设计，进行数字逻辑系统的仿真验证、时序分析、逻辑综合。它是目前应用最广泛的一种硬件描述语言。

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(1) 硬件描述语言简介——Verilog建模

模块（module）是Verilog 的基本描述单位，用于描述某个设计的功能或结构及与其他模块通信的外部端口。

模块在概念上可等同一个器件就如我们调用通用器件（与门、三态门等）或通用宏单元（计数器、ALU、CPU）等，因此，一个模块可在另一个模块中调用。一个电路设计可由多个模块组合而成。模块定义的一般语法结构如下：

module 模块名（端口名1，端口名2，端口名3, …）；

端口类型说明(input, output, inout);

参数定义(可选);

数据类型定义(wire, reg等);

实例化底层模块和基本门级元件;

连续赋值语句（assign）；

过程块结构（initial和always）

行为描述语句；

endmodule

说明部分

逻辑功能描述部分，
其顺序是任意的

一、《用可编程逻辑器件制作红外计数器Ⅱ》实验内容和要求

(1) 硬件描述语言简介——Verilog建模

简单Verilog HDL程序实例

mytri.v

```
module mytri (din, d_en, d_out);  
  input din;  
  input d_en;  
  output d_out;  
  
  assign d_out = d_en ? din : 'bz;  
endmodule
```

trist.v

```
module trist (din, d_en, d_out);  
  input din;  
  input d_en;  
  output d_out;  
  
  mytri u_mytri(din,d_en,d_out);  
endmodule
```

该例描述了一个三态驱动器。其中三态驱动门在模块 mytri 中描述，而在模块 trist 中调用了模块 mytri。模块 mytri 对 trist 而言相当于一个已存在的器件，在 trist 模块中对该器件进行实例化，实例化名 u_mytri。

注意：模块名与文件名要一致。

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(1) 硬件描述语言简介——Verilog建模方式

结构级建模: 就是根据逻辑电路的结构（逻辑图），实例引用Verilog HDL中内置的基本门级元件或者用户定义的元件或其他模块，来描述结构图中的元件以及元件之间的连接关系。

Verilog HDL中内置了12个基本门级元件模型，引用这些基本门级元件对逻辑图进行描述，也称为门级建模。

- 多输入门: and、nand、or、nor、xor、xnor

只有单个输出，1个或多个输入

- 多输出门: not、buf

允许有多个输出，但只有一个输入

- 三态门: buzfif0、buzfif1、notif0、notif1

有一个输出，一个数据输入和一个控制输入

- 上拉电阻pullup、下拉电阻pulldown

一、《用可编程逻辑器件制作红外计数器Ⅱ》实验内容和要求

(1) 硬件描述语言简介——Verilog建模方式

数据流建模 □一般用法如下： wire [位宽说明] 变量名1, 变量名2, ……， 变量名n;

assign 变量名=表达式;

运算符 (9类)

运算符分类	所含运算符
算术运算符	+, -, *, /, %
位运算符	~, &, , ^, ^~ or ~^
缩位运算符(单目)	&, ~&, , ~ , ^, ^~ or ~^
逻辑运算符	!, &&,
关系运算符 (双目)	<, >, <=, >=
相等与全等运算符	==, !=, ===, !==
逻辑移位运算符	<<, >>
连接运算符	{ }
条件运算符	?:

类型	符号	优先级别
取反	! ~ -(求2的补码)	最高优先级
算术	* / + -	
移位	>> <<	
关系	< <= > >=	
等于	== !=	
缩位	& ~& ^ ^~ ~	
逻辑	&& 	
条件	?:	最低优先级

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(1) 硬件描述语言简介——Verilog建模方式

行为级建模：描述数字逻辑电路的功能和算法。

在Verilog中，行为级描述主要使用由关键词initial或always定义的两种结构类型的语句。一个模块的内部可以包含多个initial或always语句。这两种语句之间的执行是并行的，即语句的执行与位置顺序无关。

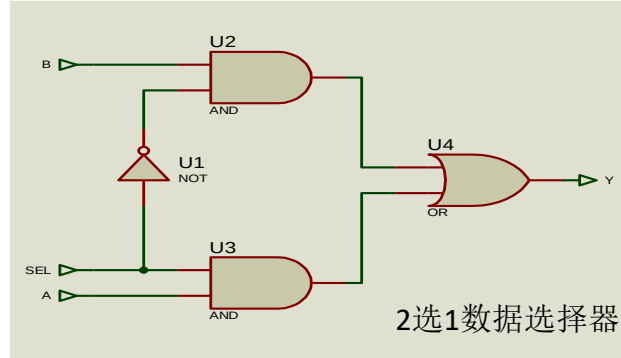
initial语句是一条初始化语句，仅执行一次，经常用于测试模块中，对激励信号进行描述，在硬件电路的行为描述中，有时为了仿真的需要，也用initial语句给寄存器变量赋初值。initial语句主要是一条面向仿真的过程语句，不能用于逻辑综合。

在always结构型语句内部有一系列过程性赋值语句，用来描述电路的功能（行为）。

一、《用可编程逻辑器件制作红外计数器Ⅱ》实验内容和要求

(1) 硬件描述语言简介——Verilog建模方式

数据流建模



```
module mux2to1_GL(a, b, sel, y);  
    input a,b,sel;  
    output y;  
  
    wire nsel,a1,b1; //定义中间变量  
  
    not U1(nsel,sel);  
    and U2(b1,b,nsel);  
    and U3(a1,a,sel);  
    or U4(y,a1,b1);  
endmodule
```

结构建模

```
module mux2to1_DF(a, b, sel, y);  
    input a,b,sel;  
    output y;  
  
    assign y = (a & sel) | (b & (~sel)); //y = sel ? a : b;  
endmodule
```

```
module mux2to1_BH(a, b, sel, y);  
    input a,b,sel;  
    output y;  
  
    reg y;  
  
    always @(*)  
        begin  
            if(sel == 1'b1)  
                y = a;  
            else  
                y = b;  
        end  
endmodule
```

行为建模

一、《用可编程逻辑器件制作红外计数器Ⅱ》实验内容和要求

(1) 硬件描述语言简介——Verilog中常量的基数表示法

整型数可以使用普通的十进制数格式（默认32位）表示，也可以用基数表示法表示，这种形式的整数格式为：

`[size]'base value`

`size` 定义以二进制位计的常量的位长；`base` 为`o` 或`O`（表示八进制），`b` 或`B`（表示二进制），`d` 或`D`（表示十进制），`h`或`H`（表示十六进制）之一；`value`是基于`base`的值的数字序列。值`x`（不定值）和`z`（高阻）以及十六进制中的`a`到`f`不区分大小写。下面是一些具体实例：

`5'O37` 5位八进制数（二进制 11111）

`4'D2` 4位十进制数（二进制0010）

`8'b0010_1x01` 8位二进制数，可用下划线分节

`7'Hx` 7位`x`（扩展的`x`），即xxxxxxx

基数格式计数形式的数通常为无符号数。这种形式的整型数的长度定义是可选的。如果没有定义一个整数型的长度，数的长度为相应值中定义的位数。下面是两个例子：

`'o721` 9位八进制数

`'hAF` 8位十六进制数

如果定义的长度比为常量指定的长度长，通常在左边填0补位。但是如果数最左边一位为`x`或`z`，就相应地用`x`或`z`在左边补位。例如：

`10'b10` 左边添0占位，0000000010

`10'bx0x1` 左边添`x`占位，xxxxxxx0x1

一、《用可编程逻辑器件制作红外计数器Ⅱ》实验内容和要求

(1) 硬件描述语言简介——Verilog中的wire型和reg型变量

wire型变量用于对结构化器件之间的物理连线的建模。如器件的管脚，内部器件如与门的输出等。wire型变量代表的是物理连接线，因此它不存储逻辑值，必须由器件所驱动，通常由assign进行赋值。如：`assign A = B ^ C;`

wire型变量定义语法如下：

```
wire [msb:lsb] wire1, wire2, ..., wireN;
```

msb 和lsb 定义了范围，并且均为常数值表达式。范围定义是可选的，如果没有定义范围，缺省值为1 位寄存器。例如：

```
wire [3:0] a;    //4位wire型数据
```

```
wire b;         //1位wire型数据
```

```
wire [4:1] c;    //4位wire型数据
```

reg型变量通常用于对存储单元的描述，如D触发器、ROM等。reg型变量在某种触发机制下分配一个值，在分配下一个值之前保留原值。但必须注意的是，reg 类型的变量，不一定是存储单元，如在always 语句中进行描述的必须用reg 类型的变量。

reg 类型定义语法如下：

```
reg [msb:lsb] reg1, reg2, ..., regN;
```

例：

```
reg [3:0] Sat;
```

```
reg cnt;
```

一、《用可编程逻辑器件制作红外计数器Ⅱ》实验内容和要求

(1) 硬件描述语言简介——Verilog中的连续赋值语句

数据流的描述是采用连续赋值语句(assign)语句来实现的。连续赋值语句用于组合逻辑的建模。例：

```
wire [3:0] Z, Preset, Clear; //线网说明
```

```
assign Z = Preset & Clear; //连续赋值语句
```

```
wire Cout, Cin ;
```

```
wire [3:0] Sum, A, B;
```

```
...
```

```
assign {Cout, Sum} = A + B + Cin;
```

```
assign Mux = (S == 3) ? D : 'bz;
```

等式左边是wire类型的变量。等式右边可以是常量、由运算符如逻辑运算符、算术运算符参与的表达。

注意如下几个方面：

1. 连续赋值语句的执行是：只要右边表达式任一个变量有变化，表达式立即被计算，计算的结果立即赋给左边信号。
2. 连续赋值语句之间是并行语句，因此与位置顺序无关。
3. “=” 用于阻塞的赋值，凡是在组合逻辑（如在assign语句中）赋值的用阻塞式赋值。

一、《用可编程逻辑器件制作红外计数器Ⅱ》实验内容和要求

(1) 硬件描述语言简介——Verilog中的过程赋值语句

Verilog提供initial和always两种过程赋值语句来实现行为的建模。这两种语句之间的执行是并行的。

通常与顺序语句块（begin end）相结合，语句块中的执行是按顺序执行的。

always语句是被重复执行的。用always语句描述硬件电路的逻辑功能时，在always语句中@符号之后紧跟着“事件控制表达式”。

逻辑电路中的敏感事件通常有两种类型：电平敏感事件和边沿触发事件。

时序逻辑建模（边沿触发事件）

```
例1:  
always @( posedge Clk or posedge Rst )  
begin  
    if(Rst)  
        Q <= 'b0;  
    else  
        Q <= D;
```

always后括号内的内容称为敏感变量，即整个always语句当敏感变量有变化时被执行，否则不执行。

因此，当Rst为1时，Q被复位，在时钟上升沿时，D被采样到Q。

时序逻辑器件的赋值语句采用非阻塞赋值“<=”

组合逻辑建模（电平敏感事件）

例2（二选一多路复用器）：

```
always @( sel, a, b )  
  
C = sel ? a : b;
```

sel、a、b是电平敏感变量，当sel、a、b中任意一个电平发生变化，后面的过程赋值语句将执行一次。

当sel为1，C被赋值为a，否则为b。

组合逻辑器件的赋值语句采用阻塞赋值“=”，且敏感变量必须写全。

一、《用可编程逻辑器件制作红外计数器Ⅱ》实验内容和要求

(1) 硬件描述语言简介——Verilog中的乘法和除法

在Verilog中，相乘表达式能够综合，会调用厂商提供的库中的乘法器，消耗较多逻辑资源。

在Verilog HDL语言中虽然有除的运算指令，但是除运算符中的除数必须是2的幂，因此无法实现除数为任意整数的除法，很大程度上限制了它的使用领域。并且多数综合工具对于除运算指令不能综合出令人满意的结果，有些甚至不能给予综合。即使可以综合，也需要比较多的资源。一般采用近似方法代替：

1. 除以2的n次方时，可以采用丢位的方法，比如a除以2，可以写成a[3:1]。
2. 一般的除法，比如a/b，都会转成乘法来做，如a*(1/b)，其中1/b的分子可以放大后再做计算。

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(1) 硬件描述语言简介——Verilog中的if条件语句

条件语句就是根据判断条件是否成立，确定下一步的运算。

Verilog语言中有3种形式的if语句：

- (1) `if(condition_expr) true_statement;`
- (2) `if(condition_expr) true_statement;`
`else fale_statement;`
- (3) `if(condition_expr1) true_statement1;`
`else if (condition_expr2) true_statement2;`
`else if (condition_expr3) true_statement3;`
`.....`
`else default_statement;`

if后面的条件表达式一般为逻辑表达式或关系表达式。执行if语句时，首先计算表达式的值，若结果为0、x或z，按“假”处理；若结果为1，按“真”处理，并执行相应的语句。

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(1) 硬件描述语言简介——Verilog中的for和while循环

软件循环语句（如在C语言中）含义是一个指令流被多次运行，在硬件描述语言中不是这样。

for循环会被展开并综合成重复的硬件结构，建议用在大量重复性的信号赋值或模块声明代码中。

while循环次数不确定，无法确定最终的硬件结构，因此不可综合，会报如“loop with non-constant loop condition must terminate within 250”的错误，一般用在测试用例的代码中。

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(1) 硬件描述语言简介——Verilog中的case分支选择语句

Case语句是一种多分支条件选择语句，一般形式如下：

```
case(case_expr)
  item_expr1: statement1;
  item_expr2: statement2;
  .....
  default: default_statement;
Endcase
```

注意：当分支项中的语句是多条语句，必须在最前面写上关键词**begin**，在最后写上关键词**end**，成为顺序语句块。

另外，用关键词**casex**和**casez**表示含有无关项**x**和高阻**z**的情况。

书写建议：**case** 的缺省项必须写，防止产生锁存器。

例：

```
case (HEX)
  4'b0001 : LED = 7'b1111001; // 1
  4'b0010 : LED = 7'b0100100; // 2
  4'b0011 : LED = 7'b0110000; // 3
  4'b0100 : LED = 7'b0011001; // 4
  4'b0101 : LED = 7'b0010010; // 5
  4'b0110 : LED = 7'b0000010; // 6
  4'b0111 : LED = 7'b1111000; // 7
  4'b1000 : LED = 7'b0000000; // 8
  4'b1001 : LED = 7'b0010000; // 9
  4'b1010 : LED = 7'b0001000; // A
  4'b1011 : LED = 7'b0000011; // B
  4'b1100 : LED = 7'b1000110; // C
  4'b1101 : LED = 7'b0100001; // D
  4'b1110 : LED = 7'b0000110; // E
  4'b1111 : LED = 7'b0001110; // F
  default : LED = 7'b1000000; // 0
endcase
```


一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

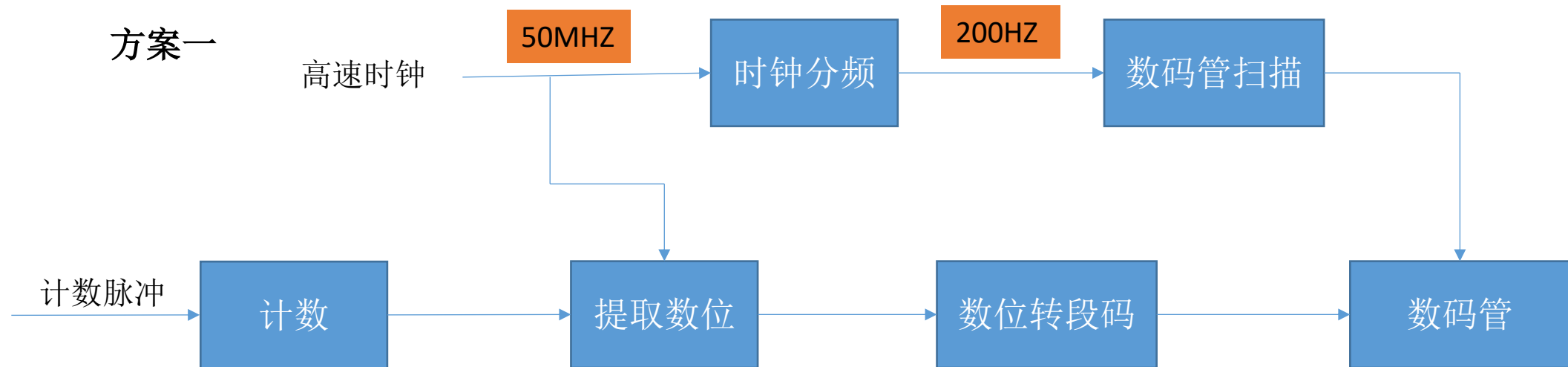
(1) 硬件描述语言简介——Verilog保留字

always and assign begin buf buf if0 bufif1 case casex casez cmos deassign default
defparam disable edge else end endcase endmodule endfunction endprimitive endspecify
endtable endtask event for force forever fork function highz0 highz1 if ifnone
initial inout input integer join large macrmodule medium module nand negedge nmos nor
not notif0 notif1 or output parameter pmos posedge primitive pull0 pull1 pullup pulldown
rcmos real realtime reg release repeat rnmos rpmos rtran rtranif0 rtranif1 scalared small
specify specparam strong0 strong1 supply0 supply1 table task time trantranif0 tranif1 tri
tri0 tri1 triand trior trireg vectored wait wand weak0 weak1 while wire wor xnor xor

不要用作变量和输入输出引脚名

一、《用可编程逻辑器件制作红外计数器Ⅱ》实验内容和要求

(2) 计数器功能实现——设计框图



一、《用可编程逻辑器件制作红外计数器Ⅱ》实验内容和要求

(2) 计数器功能实现——模块输入输出与计数

模块名与文件名要一致



```
module IR_counter(  
    input clk_50m,  
    input rst_n,  
    input pulse_in,  
    output reg [3:0] dig_sel,  
    output reg [7:0] seg_code;  
    output reg alarm);  
  
endmodule
```

```
reg [7:0] dig1_segcode;  
reg [7:0] dig2_segcode;  
reg [7:0] dig3_segcode;  
reg [7:0] dig4_segcode;  
reg [3:0] dig1;  
reg [3:0] dig2;  
reg [3:0] dig3;  
reg [3:0] dig4;  
reg [3:0] dig1_tmp;  
reg [3:0] dig2_tmp;  
reg [3:0] dig3_tmp;  
reg [3:0] dig4_tmp;  
parameter num0_segcode = 8'b0000_0011;//a b c d e f g dp 低有效  
parameter num1_segcode = 8'b1001_1111;  
parameter num2_segcode = 8'b0010_0101;  
parameter num3_segcode = 8'b0000_1101;  
parameter num4_segcode = 8'b1001_1001;  
parameter num5_segcode = 8'b0100_1001;  
parameter num6_segcode = 8'b0100_0001;  
parameter num7_segcode = 8'b0001_1111;  
parameter num8_segcode = 8'b0000_0001;  
parameter num9_segcode = 8'b0000_1001;
```

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(2) 计数器功能实现——定义千位、百位、十位、个位及段码数据

```
reg [13:0] counter;
```

```
always @(negedge pulse_in or negedge rst_n)
```

```
begin
```

```
if(~rst_n)
```

```
begin
```

```
counter <= 14'd0;
```

```
end
```

```
else
```

```
begin
```

```
if(counter < 14'd9999)
```

```
begin
```

```
counter <= counter + 1'd1;
```

```
end
```

```
else
```

```
begin
```

```
counter <= 14'd0;
```

```
alarm <= 1'b1;
```

```
end
```

```
end
```

```
end
```



9999的值需要14位寄存器

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(2) 计数器功能实现——从计数值提取千位、百位、十位、个位

```
reg [13:0] counter_buf;
```

```
always @(posedge clk_50m or negedge rst_n)
```

```
begin
```

```
if(~rst_n)
```

```
begin
```

```
dig1 <= 4'd0;
```

```
dig2 <= 4'd0;
```

```
dig3 <= 4'd0;
```

```
dig4 <= 4'd0;
```

```
dig1_tmp <= 4'd0;
```

```
dig2_tmp <= 4'd0;
```

```
dig3_tmp <= 4'd0;
```

```
dig4_tmp <= 4'd0;
```

```
counter_buf <= 14'd0;
```

```
end
```

```
else
```

```
begin
```

```
if(counter_buf == 14'd0)
```

```
begin
```

```
dig1 <= dig1_tmp;
```

```
dig2 <= dig2_tmp;
```

```
dig3 <= dig3_tmp;
```

```
dig4 <= dig4_tmp;
```

```
dig1_tmp <= 4'd0;
```

```
dig2_tmp <= 4'd0;
```

```
dig3_tmp <= 4'd0;
```

```
dig4_tmp <= 4'd0;
```

```
counter_buf <= counter;
```

```
end
```

```
else
```

```
begin
```

```
if(counter_buf >= 14'd1000)
```

```
begin
```

```
counter_buf <= counter_buf - 14'd1000;
```

```
dig1_tmp <= dig1_tmp + 4'd1;
```

```
end
```

```
else if((counter_buf < 14'd1000) && (counter_buf >= 14'd100))
```

```
begin
```

```
counter_buf <= counter_buf - 14'd100;
```

```
dig2_tmp <= dig2_tmp + 4'd1;
```

```
end
```

```
else if((counter_buf < 14'd100) && (counter_buf >= 14'd10))
```

```
begin
```

```
counter_buf <= counter_buf - 14'd10;
```

```
dig3_tmp <= dig3_tmp + 4'd1;
```

```
end
```

```
else if((counter_buf < 14'd10) && (counter_buf >= 14'd1))
```

```
begin
```

```
counter_buf <= counter_buf - 14'd1;
```

```
dig4_tmp <= dig4_tmp + 4'd1;
```

```
end
```

```
end
```

```
end
```

```
end
```

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(2) 计数器功能实现——千位、百位段码赋值

```
always @(negedge pulse_in or negedge rst_n)
begin
    if(~rst_n)
        begin
            dig1_segcode <= num0_segcode;
            dig2_segcode <= num0_segcode;
            dig3_segcode <= num0_segcode;
            dig4_segcode <= num0_segcode;
        end
    else
        begin
            case(dig1)
                4'd0: dig1_segcode <= num0_segcode;
                4'd1: dig1_segcode <= num1_segcode;
                4'd2: dig1_segcode <= num2_segcode;
                4'd3: dig1_segcode <= num3_segcode;
                4'd4: dig1_segcode <= num4_segcode;
                4'd5: dig1_segcode <= num5_segcode;
                4'd6: dig1_segcode <= num6_segcode;
                4'd7: dig1_segcode <= num7_segcode;
                4'd8: dig1_segcode <= num8_segcode;
                4'd9: dig1_segcode <= num9_segcode;
                default: dig1_segcode <= num0_segcode;
            endcase
        end
    end
```

```
case(dig2)
    4'd0: dig2_segcode <= num0_segcode;
    4'd1: dig2_segcode <= num1_segcode;
    4'd2: dig2_segcode <= num2_segcode;
    4'd3: dig2_segcode <= num3_segcode;
    4'd4: dig2_segcode <= num4_segcode;
    4'd5: dig2_segcode <= num5_segcode;
    4'd6: dig2_segcode <= num6_segcode;
    4'd7: dig2_segcode <= num7_segcode;
    4'd8: dig2_segcode <= num8_segcode;
    4'd9: dig2_segcode <= num9_segcode;
    default: dig2_segcode <= num0_segcode;
endcase
```

预习：这里如果用组合逻辑应该怎样描述？

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(2) 计数器功能实现——十位、个位段段码赋值

```
case(dig3)
  4'd0: dig3_segcode <= num0_segcode;
  4'd1: dig3_segcode <= num1_segcode;
  4'd2: dig3_segcode <= num2_segcode;
  4'd3: dig3_segcode <= num3_segcode;
  4'd4: dig3_segcode <= num4_segcode;
  4'd5: dig3_segcode <= num5_segcode;
  4'd6: dig3_segcode <= num6_segcode;
  4'd7: dig3_segcode <= num7_segcode;
  4'd8: dig3_segcode <= num8_segcode;
  4'd9: dig3_segcode <= num9_segcode;
  default: dig3_segcode <= num0_segcode;
endcase
```

```
case(dig4)
  4'd0: dig4_segcode <= num0_segcode;
  4'd1: dig4_segcode <= num1_segcode;
  4'd2: dig4_segcode <= num2_segcode;
  4'd3: dig4_segcode <= num3_segcode;
  4'd4: dig4_segcode <= num4_segcode;
  4'd5: dig4_segcode <= num5_segcode;
  4'd6: dig4_segcode <= num6_segcode;
  4'd7: dig4_segcode <= num7_segcode;
  4'd8: dig4_segcode <= num8_segcode;
  4'd9: dig4_segcode <= num9_segcode;
  default: dig4_segcode <= num0_segcode;
endcase
end
end
```

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(2) 计数器功能实现——高速时钟分频与数码管扫描

```
reg [16:0] div_cnt;
reg div_clk;

always @(posedge clk_50m or negedge rst_n)
begin
    if(~rst_n)
    begin
        div_cnt <= 17'd0;
        div_clk <= 1'b0;
    end
    else
    begin
        if(div_cnt < 17'd125000)
        begin
            div_cnt <= div_cnt + 1'd1;
        end
        else
        begin
            div_cnt <= 17'd0;
            div_clk <= ~div_clk;
        end
    end
end
```

50M分到200Hz

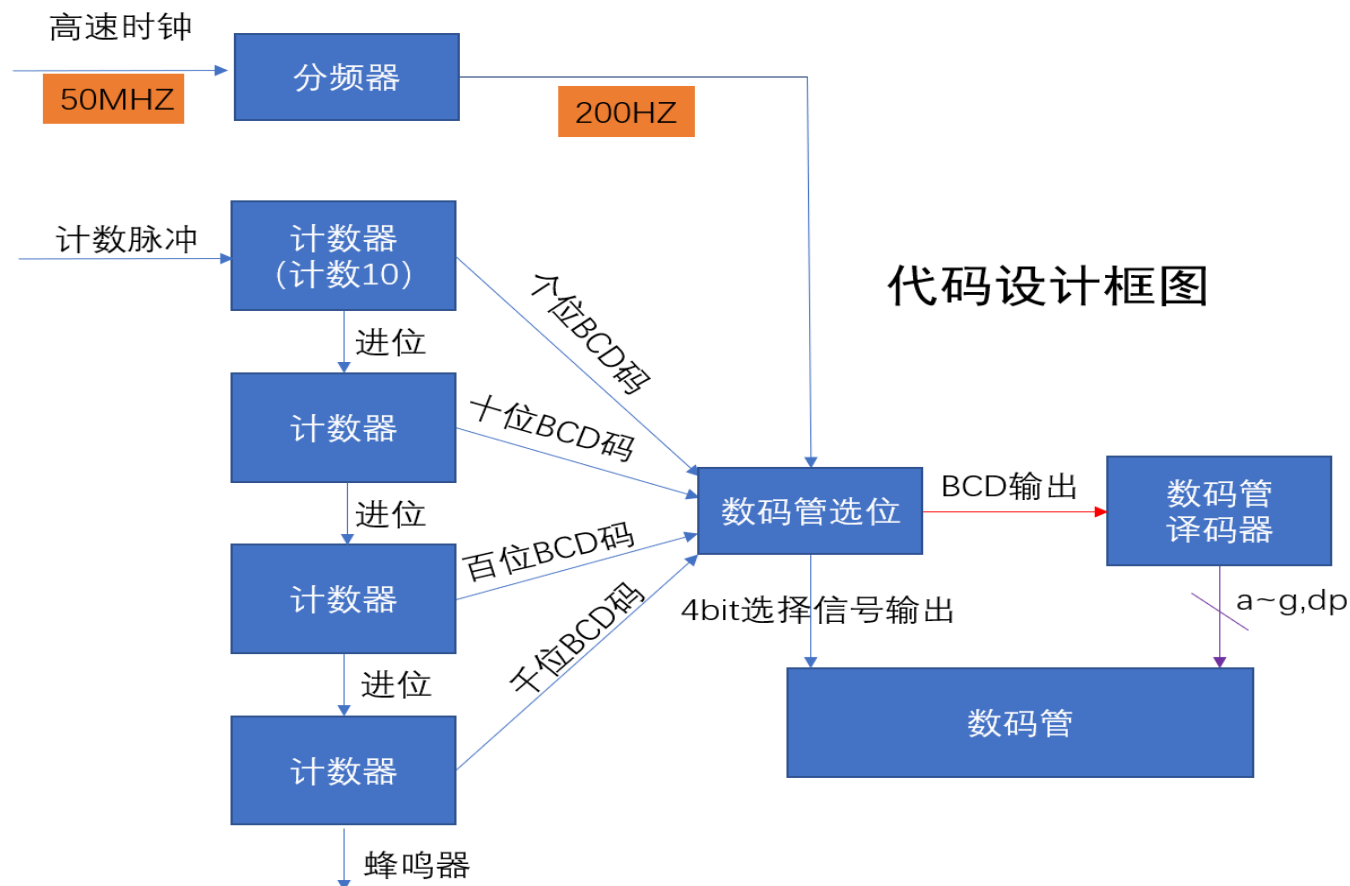
```
always @(posedge div_clk or negedge rst_n)
begin
    if(~rst_n)
    begin
        dig_sel <= 4'b0111;
        seg_code <= dig1_segcode;
    end
    else
    begin
        case(dig_sel)
            4'b0111: begin dig_sel <= 4'b1011; seg_code <= dig2_segcode; end
            4'b1011: begin dig_sel <= 4'b1101; seg_code <= dig3_segcode; end
            4'b1101: begin dig_sel <= 4'b1110; seg_code <= dig4_segcode; end
            4'b1110: begin dig_sel <= 4'b0111; seg_code <= dig1_segcode; end
            default: begin dig_sel <= 4'b0111; seg_code <= dig1_segcode; end
        endcase
    end
end
```

第一位千位显示起，所以引脚dig_sel[3]是PIN _ 99(SM SEL0)

一、《用可编程逻辑器件制作红外计数器Ⅱ》实验内容和要求

(2) 计数器功能实现——设计框图

方案二



一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(2) 计数器功能实现——模十计数器计数

```
reg[2:0]CP;
always @(negedge pulse_in or negedge rst_n)
begin
    if(~rst_n)
        begin
            dig[0] <= 4'd0;
            CP[0] <= 1'd0;
        end
    else if(dig[0]<4'd9) begin
        dig[0] <= dig[0]+4'd1;
        CP[0] <= 1'd0;
    end
    else
        begin
            dig[0] <= 4'd0;
            CP[0] <= 1'd1;
        end
end
```

```
always @(posedge CP[0] or negedge rst_n)
begin
    if(~rst_n)
        begin
            dig[1] <= 4'd0;
            CP[1] <= 1'd0;
        end
    else if(dig[1]<4'd9)
        begin
            dig[1] <= dig[1]+4'd1;
            CP[1] <= 1'd0;
        end
    else
        begin
            dig[1] <= 4'd0;
            CP[1] <= 1'd1;
        end
end
```

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(2) 计数器功能实现——十位、个位段段码赋值

```
always @(posedge CP[1] or negedge rst_n)
begin
    if(~rst_n)
        begin
            dig[2] <= 4'd0;
            CP[2] <= 1'd0;
        end
    else if(dig[2]<4'd9)
        begin
            dig[2] <= dig[2]+4'd1;
            CP[2] <= 1'd0;
        end
    else
        begin
            dig[2] <= 4'd0;
            CP[2] <= 1'd1;
        end
end
```

```
always @(posedge CP[2] or negedge rst_n)
begin
    if(~rst_n)
        begin
            dig[3] <= 4'd0;
            alarm <= 1'd0;
        end
    else if(dig[3]<4'd9)
        begin
            dig[3] <= dig[3]+4'd1;
            alarm <= 1'd0;
        end
    else
        begin
            dig[3] <= 4'd0;
            alarm <= 1'd1;
        end
end
```

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(2) 计数器功能实现——数码管选位和显示

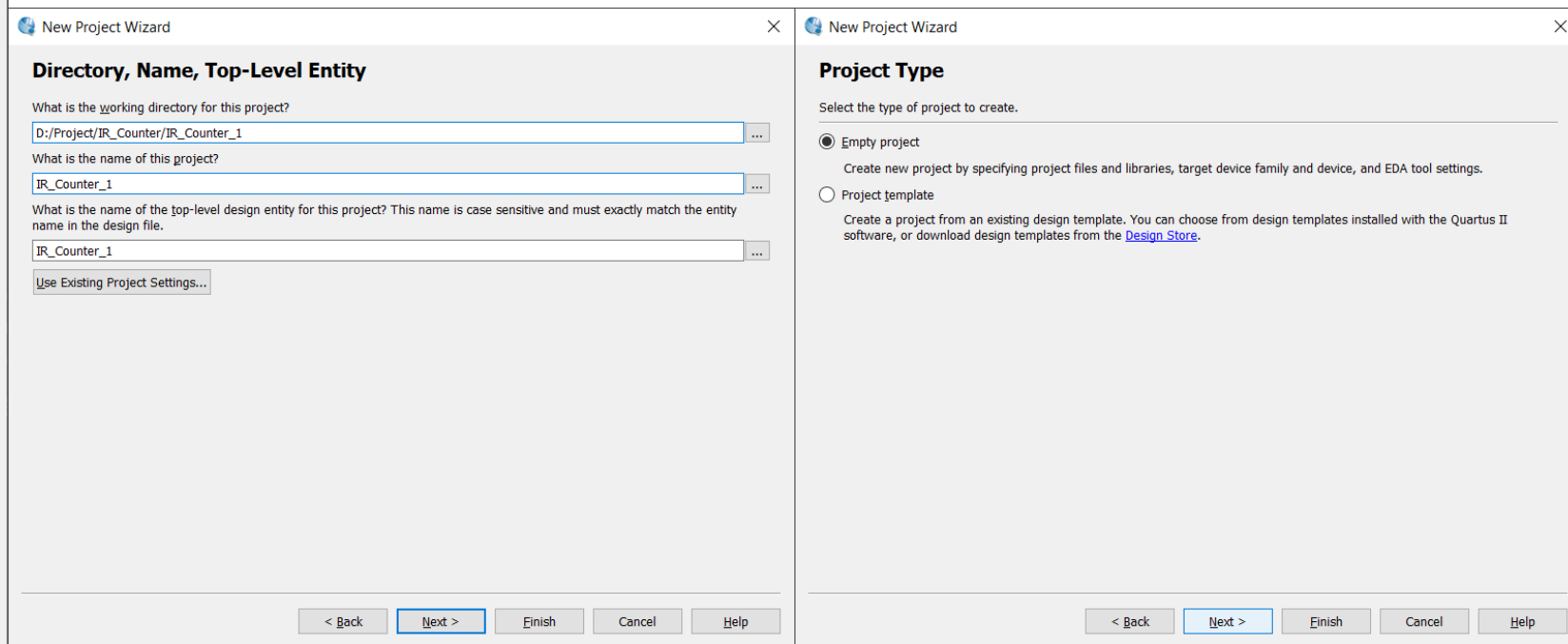
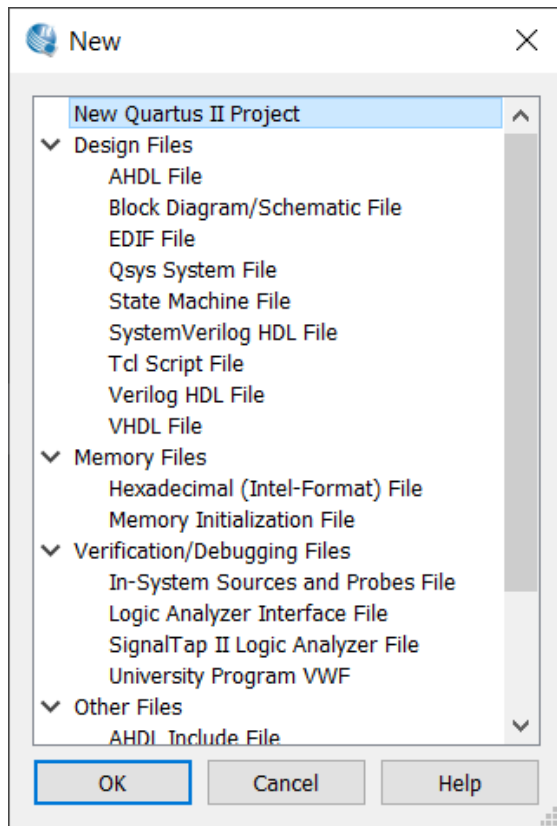
```
always @(posedge div_clk or negedge rst_n)
begin
    if(~rst_n)
        begin
            dig_sel <= 4'b0111;
            dig_reg <= dig[3];
        end
    else
        begin
            case(dig_sel)
                4'b0111: begin dig_sel <= 4'b1011; dig_reg <= dig[2]; end
                4'b1011: begin dig_sel <= 4'b1101; dig_reg <= dig[1]; end
                4'b1101: begin dig_sel <= 4'b1110; dig_reg <= dig[0]; end
                4'b1110: begin dig_sel <= 4'b0111; dig_reg <= dig[3]; end
                default: begin dig_sel <= 4'b0111; dig_reg <= dig[3]; end
            endcase
        end
    end
end
```

第一位个位显示起，同前面的区别

```
always @(*)
begin
    if(~rst_n)
        begin
            seg_code <= 8'b0000_0011;
        end
    else
        begin
            case(dig_reg)
                4'd0: seg_code <= num0_segcode;
                4'd1: seg_code <= num1_segcode;
                4'd2: seg_code <= num2_segcode;
                4'd3: seg_code <= num3_segcode;
                4'd4: seg_code <= num4_segcode;
                4'd5: seg_code <= num5_segcode;
                4'd6: seg_code <= num6_segcode;
                4'd7: seg_code <= num7_segcode;
                4'd8: seg_code <= num8_segcode;
                4'd9: seg_code <= num9_segcode;
                default: seg_code <= num0_segcode;
            endcase
        end
    end
end
```

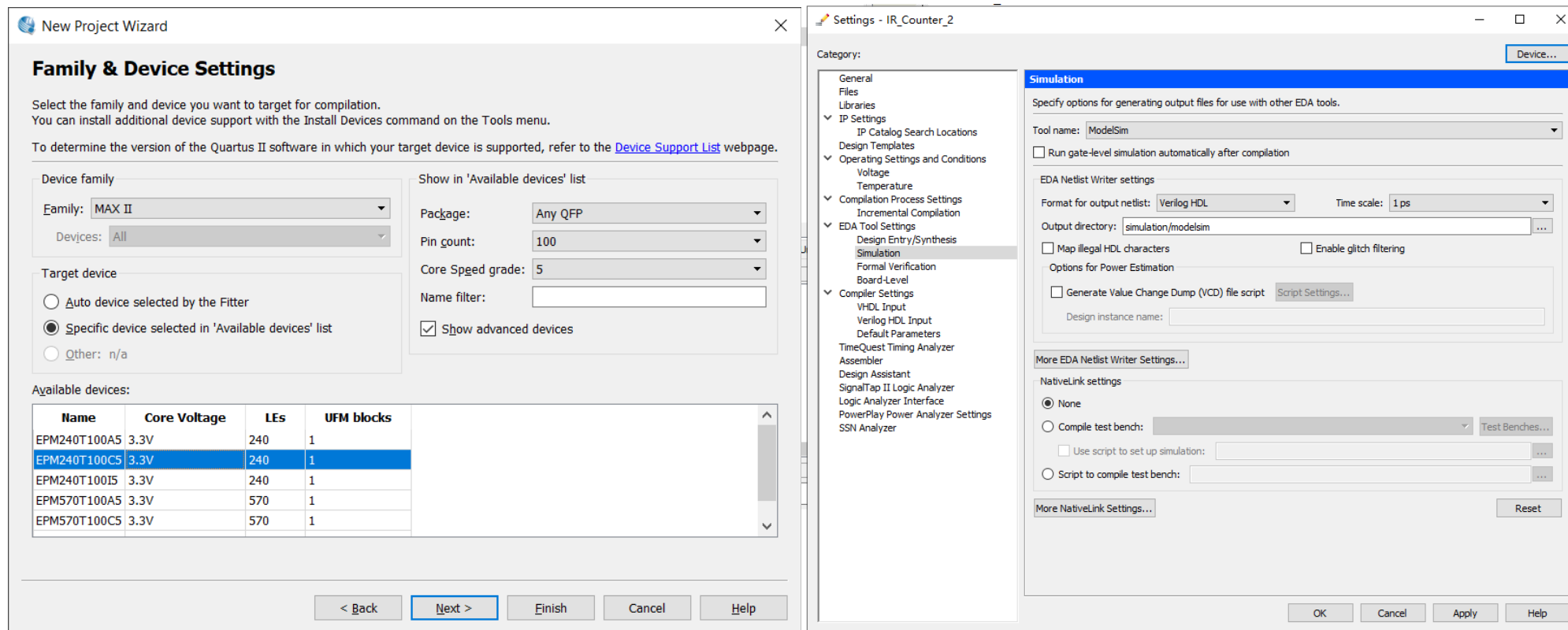
一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(3) Quartus II——新建工程及文件



一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(3) Quartus II——新建工程及文件

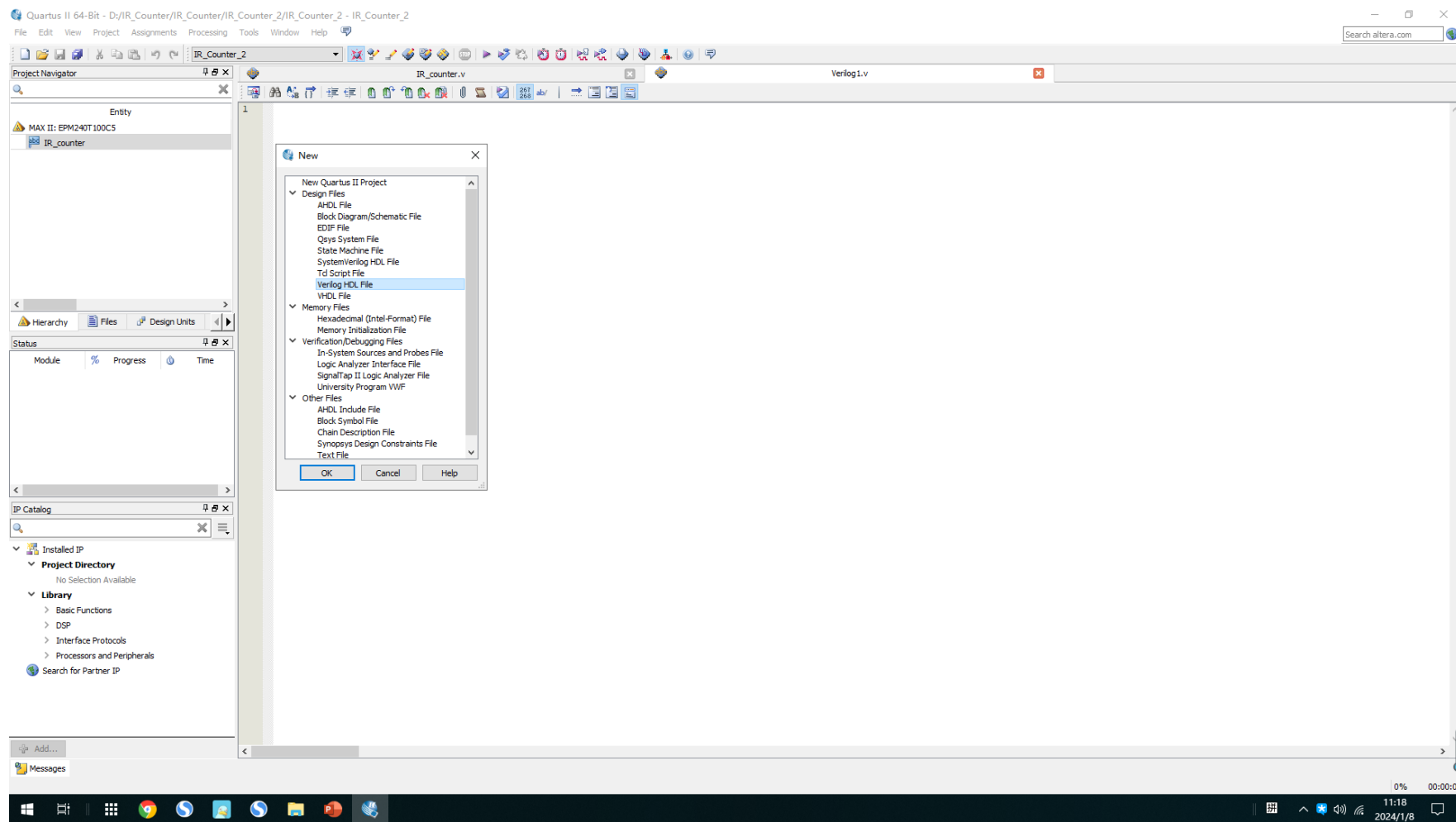


选择器件类型和型号

选择仿真软件和仿真语言，这里选择 verilog/Modelsim

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(3) Quartus II——新建工程及文件



新建文件并编写代码

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(3) Quartus II——编译与优化

Report - IR_Counter_1	
Flow Summary	
Flow Status	Successful - Sun Apr 23 15:51:05 2023
Quartus II 64-Bit Version	14.1.0 Build 186 12/03/2014 SJ Full Version
Revision Name	IR_Counter_1
Top-level Entity Name	IR_Counter_1
Family	MAX II
Device	EPM240T100C5
Timing Models	Final
Total logic elements	116 / 240 (48 %)
Total pins	16 / 80 (20 %)
Total virtual pins	0
UFM blocks	0 / 1 (0 %)

调用74符号库

Report - IR_Counter_2	
Flow Summary	
Flow Status	Successful - Mon Apr 24 18:40:08 2023
Quartus II 64-Bit Version	14.1.0 Build 186 12/03/2014 SJ Full Version
Revision Name	IR_Counter_2
Top-level Entity Name	IR_counter
Family	MAX II
Device	EPM240T100C5
Timing Models	Final
Total logic elements	236 / 240 (98 %)
Total pins	16 / 80 (20 %)
Total virtual pins	0
UFM blocks	0 / 1 (0 %)

使用硬件描述语言

实现同样功能，使用硬件描述语言比调用74符号库消耗更多逻辑资源，根源是本次实验大量使用时序逻辑，而上次实验以组合逻辑为主（译码器、多路复用器均是组合逻辑）。事实上，本次实验方案一需开启优化。

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(3) Quartus II——编译与优化

Quartus II 64-Bit - D:/Project/IR_Counter/IR_C

File Edit View Project Assignments Processir

Device... Settings...

Settings - IR_Counter_2

Category:

- General
- Files
- Libraries
- IP Settings
 - IP Catalog Search Locations
 - Design Templates
- Operating Settings and Conditions
 - Voltage
 - Temperature
- Compilation Process Settings
 - Incremental Compilation
- EDA Tool Settings
 - Design Entry/Synthesis
 - Simulation
 - Formal Verification
 - Board-Level
- Compiler Settings 1**
 - VHDL Input
 - Verilog HDL Input
 - Default Parameters
 - TimeQuest Timing Analyzer
 - Assembler
 - Design Assistant
 - SignalTap II Logic Analyzer

Compiler Settings

Specify high-level optimization settings for the Compiler (including integrated synthesis and fitting). These settings control the optimization focus and algorithms that will be performed throughout design compilation.

Optimization mode

- ☒ Balanced (Normal flow)
- ☐ Performance (High effort - increases runtime)
- ☐ Performance (Aggressive - increases runtime and area)
- ☐ Power (High effort - increases runtime)
- ☐ Power (Aggressive - increases runtime, reduces performance)
- ☐ Area (Aggressive - reduces performance)

Prevent register optimizations

- ☐ Prevent register merging
- ☐ Prevent register duplication
- ☐ Prevent register retiming

Advanced Settings (Synthesis)... **Advanced Settings (Fitter)... 2**

Advanced Fitter Settings

Specify the settings for the logic options in your project. Assignments made to an individual node or entity in the Assignment Editor will override the option settings in this dialog box.

<<Search>> Show: All

Name:	Setting:
ALM Register Packing Effort	Medium
Auto Delay Chains	On
Auto Delay Chains for High Fanout Input Pins	Off
Auto Fit Effort Desired Slack Margin	0 ns
Auto Global Clock	On
Auto Global Register Control Signals	On
Auto Packed Registers 3	Minimize Area
Auto Register Duplication	Auto
Enable Bus-Hold Circuitry	Off

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(3) Quartus II——添加管脚约束

The screenshot displays the Quartus II software interface during the compilation of a project named "IR_Counter_2". The main window shows the "Flow Summary" tab, indicating a successful compilation on Monday, January 08, 2024, at 11:24:27. The device used is EPM240T100C5, and the total logic elements are 252. The pin assignment summary is shown in the "Pin Legend" window, listing various pins and their assigned functions.

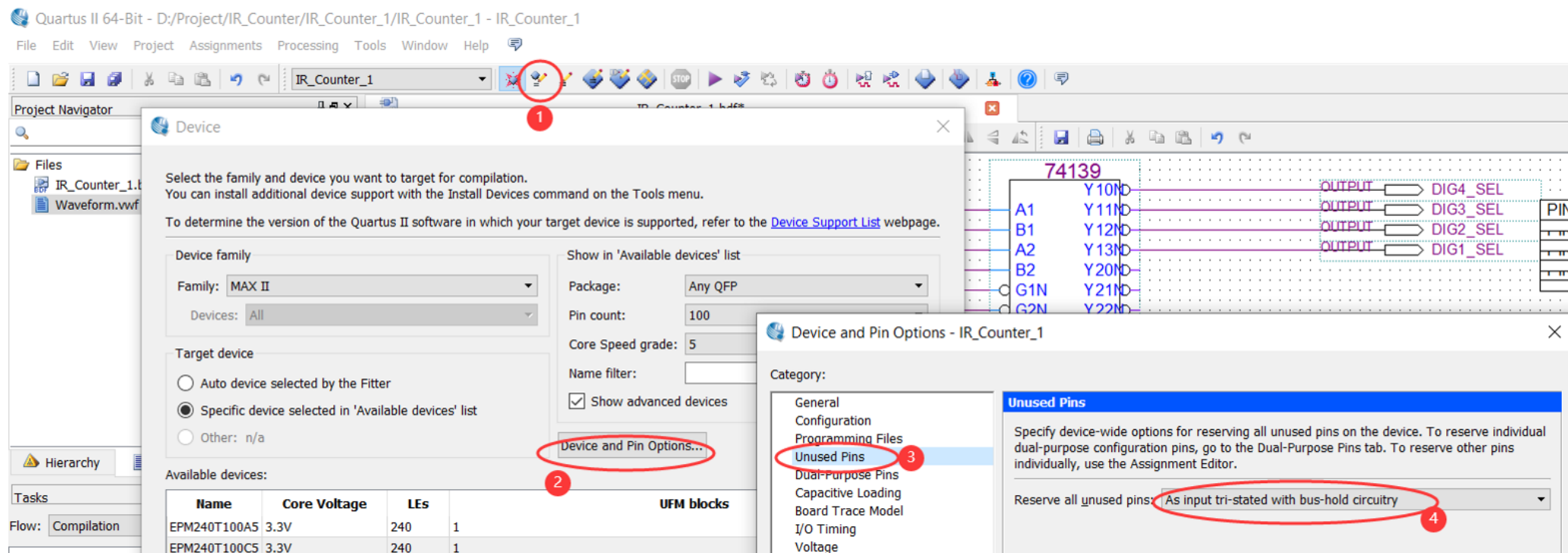
Pin Legend:

Node Name	Direction	Location	I/O Bank	I/O Standard
alarm	Output	PIN_86	2	3.3-V LVCMOS
clk_50m	Input	PIN_12	1	3.3-V LVCMOS
dig_sel[3]	Output	PIN_99	2	3.3-V LVCMOS
dig_sel[2]	Output	PIN_97	2	3.3-V LVCMOS
dig_sel[1]	Output	PIN_92	2	3.3-V LVCMOS
dig_sel[0]	Output	PIN_90	2	3.3-V LVCMOS
pulse_in	Input	PIN_39	1	3.3V Sch...er Input
rst_n	Input	PIN_40	1	3.3V Sch...er Input
seg_code[7]	Output	PIN_100	2	3.3-V LVCMOS
seg_code[6]	Output	PIN_98	2	3.3-V LVCMOS
seg_code[5]	Output	PIN_96	2	3.3-V LVCMOS
seg_code[4]	Output	PIN_95	2	3.3-V LVCMOS
seg_code[3]	Output	PIN_91	2	3.3-V LVCMOS
seg_code[2]	Output	PIN_89	2	3.3-V LVCMOS
seg_code[1]	Output	PIN_88	2	3.3-V LVCMOS
seg_code[0]	Output	PIN_87	2	3.3-V LVCMOS

The "Pin Legend" window also includes a legend for pin types: User I/O (white circle), User assigned I/O (red circle), Fitter assigned I/O (green circle), Unbonded pad (grey circle), Reserved pin (blue circle), DEV_OE (red circle with 'E'), DEV_CLR (red circle with 'R'), CLK_n (white circle with 'L'), TDI (white circle with 'T'), TCK (white circle with 'K'), TMS (white circle with 'M'), TDO (white circle with 'D'), VCCINT (white circle with 'V'), VCCIO (white circle with 'V'), GNDINT (white circle with 'G'), and GNDIO (white circle with 'G').

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

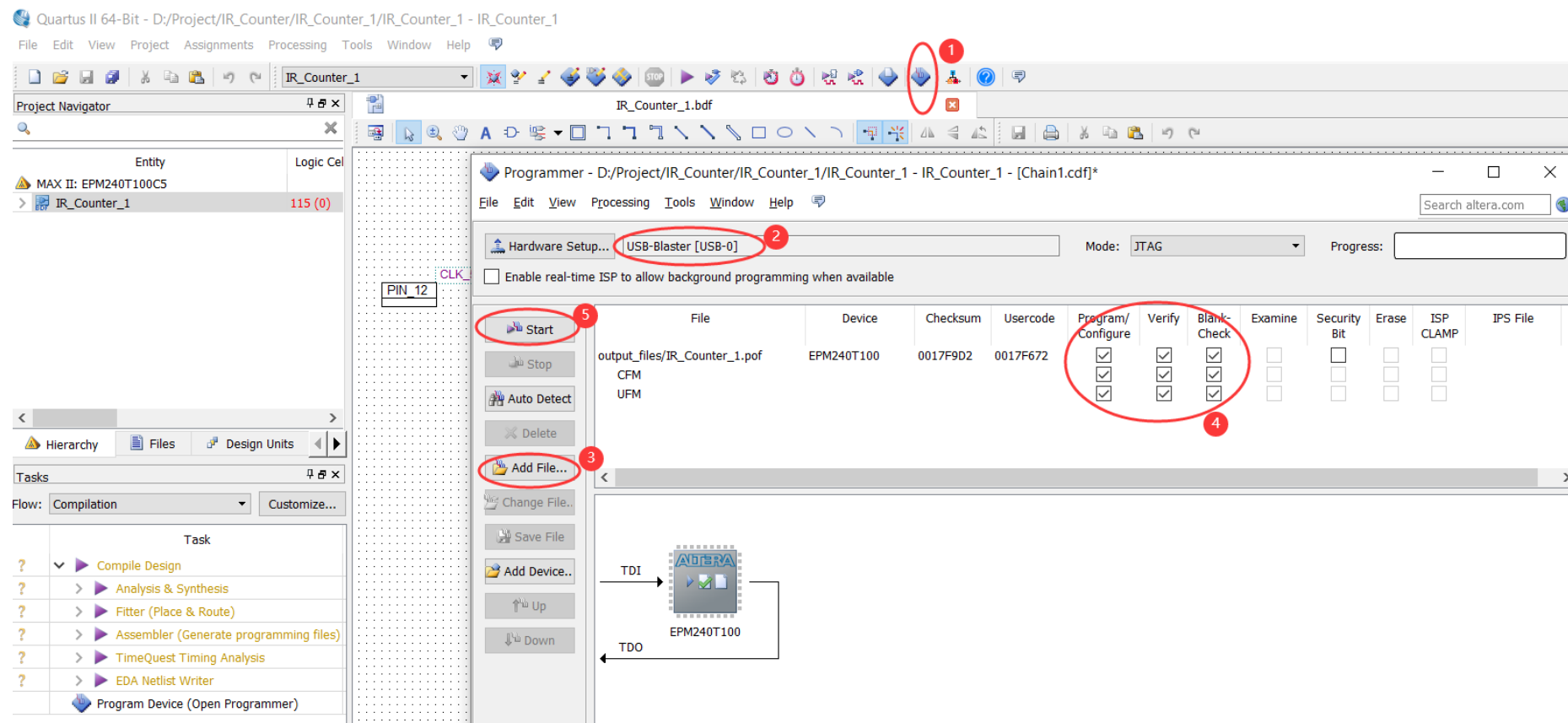
(3) Quartus II——未分配的管脚的处理方法



一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

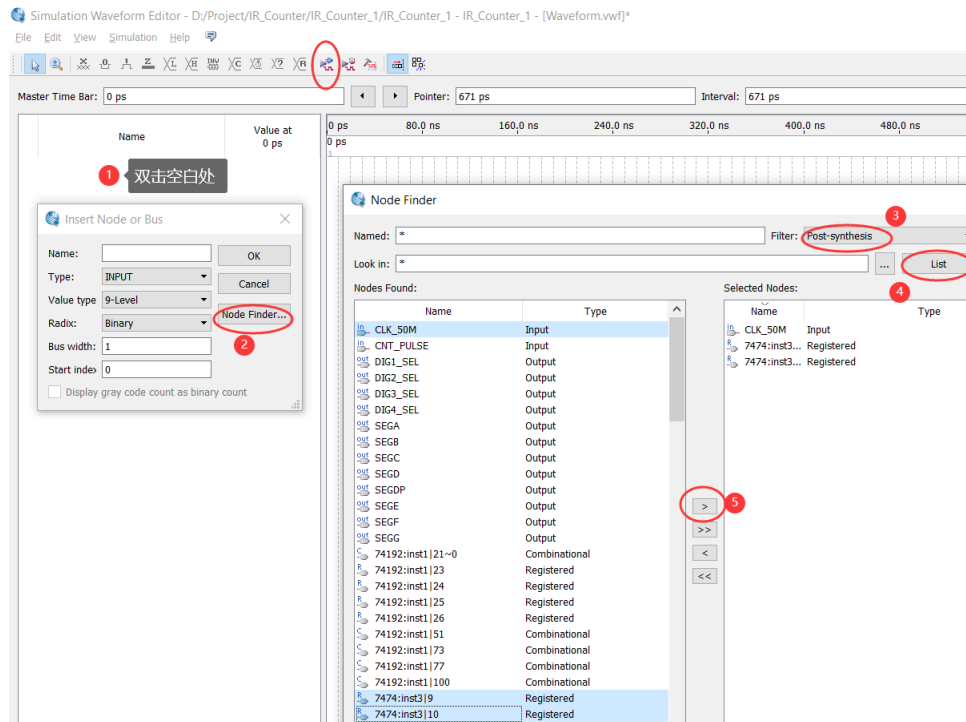
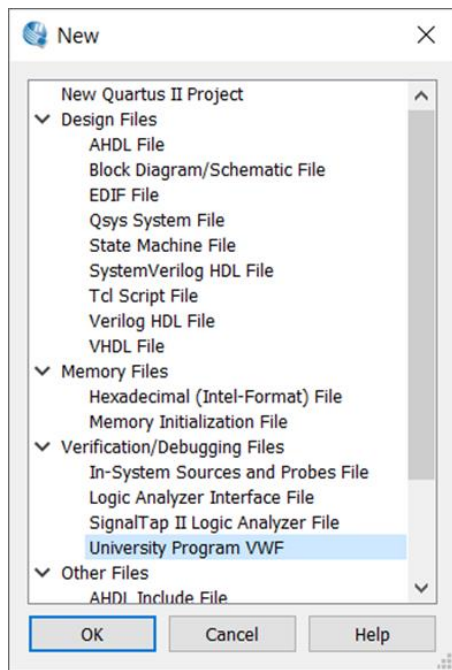
(3) Quartus II——烧录

优化和添加管脚约束后再编译一次，重新命名的才会在管脚里面，烧录后如果程序不是立马运行，先断电一下再上电。



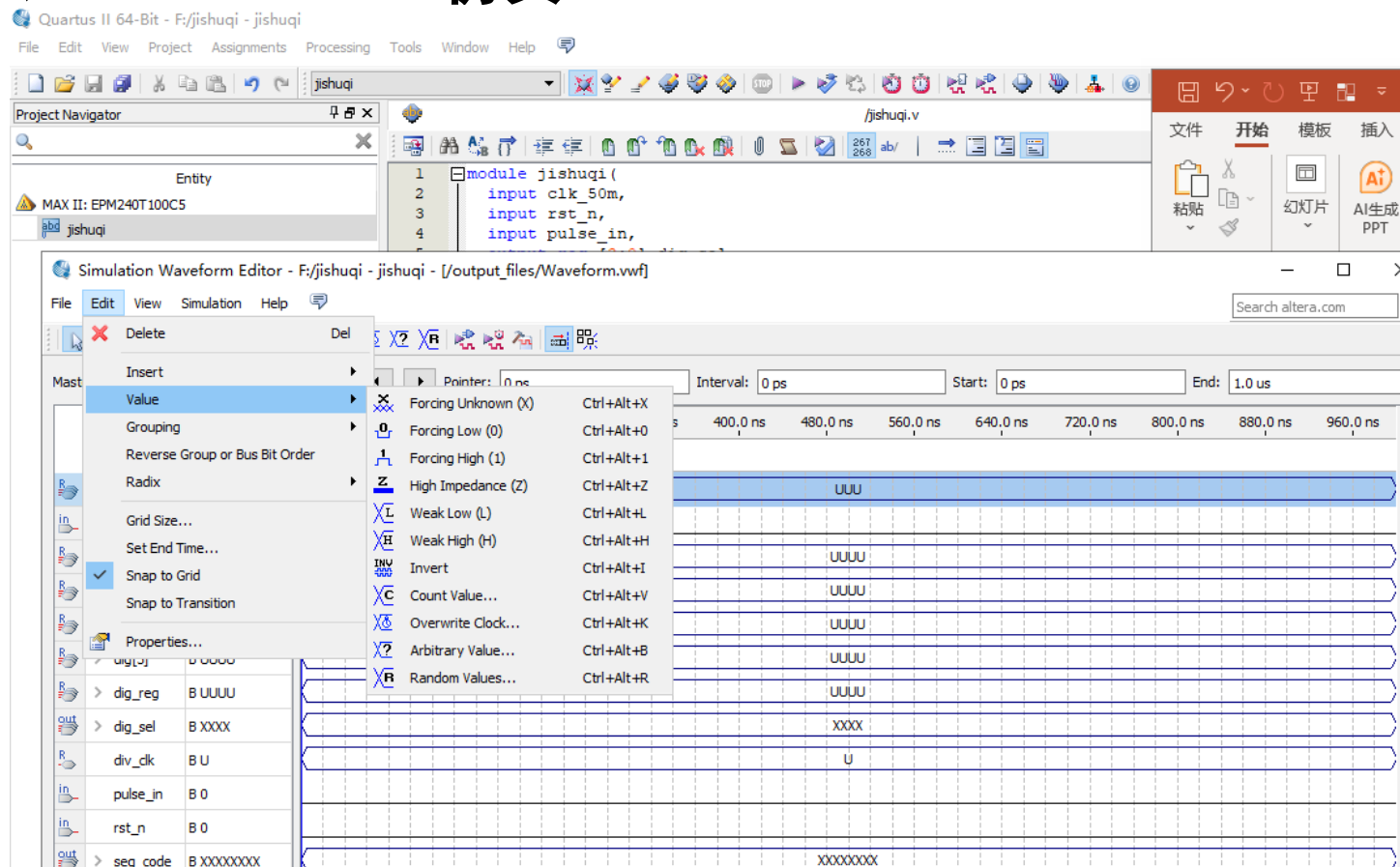
一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(3) Quartus II——仿真



一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(3) Quartus II——仿真

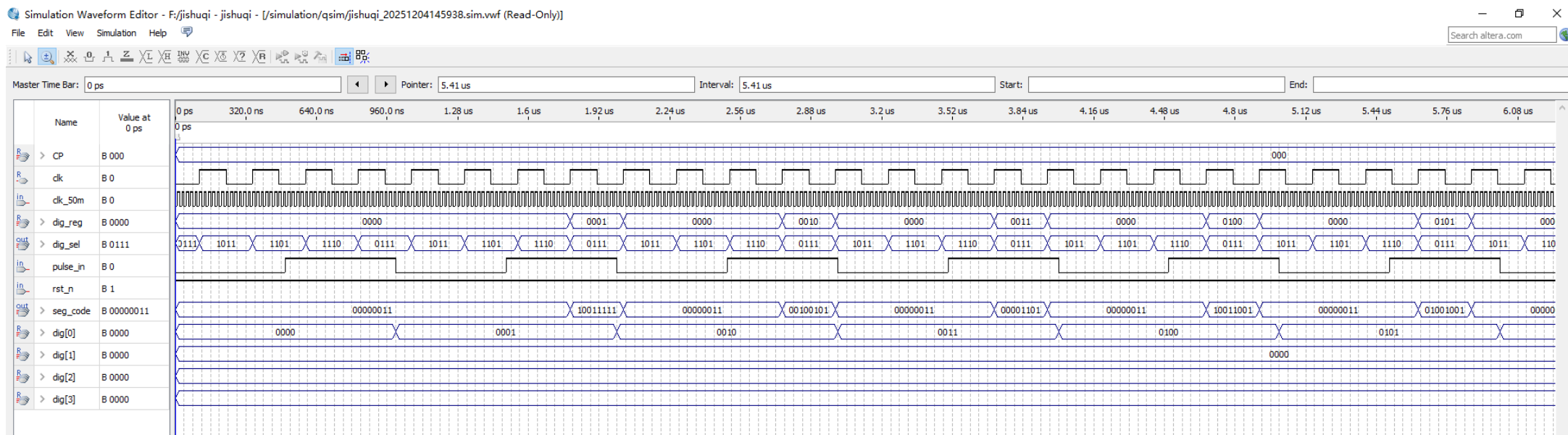


仿真前需要编译整个工程，编译后点RUN FUNCTIONAL SIMULATION。仿真界面菜单Edit-Set End Time可设置仿真时长；

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(3) Quartus II——仿真

第二种方案仿真波形参考



因为仿真时间问题，分频信号频率至少是按键信号频率的4倍，这样才能分别显示一次四位数字，源代码相关部分需要修改。

一、《用可编程逻辑器件制作红外计数器Ⅲ》实验内容和要求

(4) 实践任务

1. 在可编程逻辑器件开发板上验证所设计的电路。计数输入端使用按键，报警使用蜂鸣器。按键输入管脚可开启施密特触发器。方案一二任选一种验收。
2. 仿真练习：方案一例程中有一处bug，会导致计数器从第二个输入脉冲开始更新数码管数值，请使用仿真手段找到问题原因并解决这个bug（关注复位后按键产生第一个下降沿时采到的个位值。），验收内容需要展示排查上述bug时的仿真图；或方案二的仿真图（需要更改部分代码）。
3. 实验报告：任选一种方案的实现图片加仿真图，课后反思（你学到了哪些知识点或者重难点？发现了哪些问题，运用哪些分析方法解决的？对课程内容、流程或者其他方面有哪些想法和建议？）。

二、下节课预习内容及要求

(1) 阅读课程组所提供的学习资料，回答以下问题或完成相应任务。

1、使用仿真软件仿真下面的电路图，用虚拟示波器观察三路波形的相位关系，测量两路比较器的翻转门限电压。输入波形为正弦波，幅值2.5V（峰-峰值5V），直流偏置2.5V。（100分）

